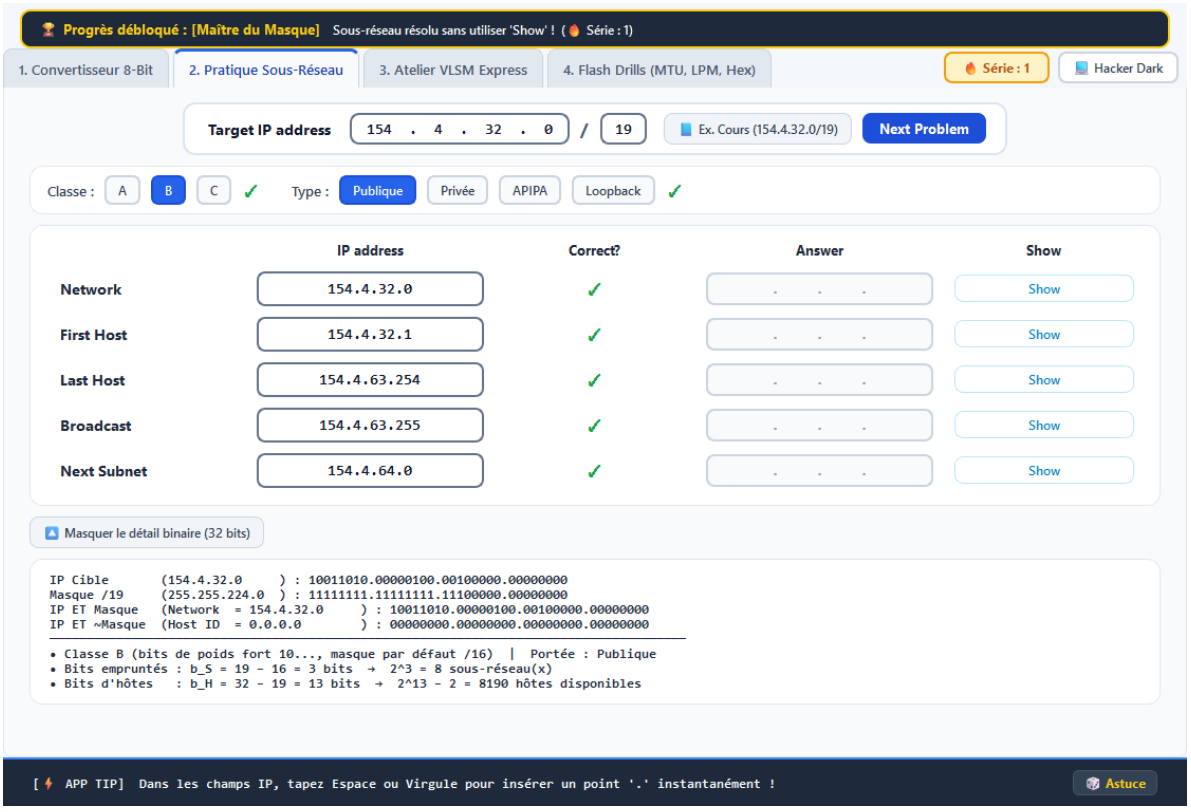
INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS (INPT)

Cursus Ingénieur — Filière ASEDS

IPv4 Subnet Toolkit v1.2.0

Manuel Pédagogique de l'Application & Raisonnement Complet du Cours de Réseau IPv4

Comprendre le rôle de chaque composant de l'interface graphique, la méthodologie de résolution visuelle et les fondements mathématiques du cours (Adressage, Masques, Sous-Réseaux, VLSM, Fragmentation MTU, Routage LPM & Checksum 16 bits).



Version 1.2.0 — Octobre 2026

Document de référence illustré par les captures d'écran réelles de l'application

Contents

1 Philosophie Pédagogique : Pourquoi l'Application est Conçue Ainsi **2**
1.1 Pilier 1 — « Tester la Décision, Automatiser la Répétition » . . . 2
1.2 Pilier 2 — La Divulgation Progressive (Tiroir Binaire 32-Bit) . . . 2
1.3 Pilier 3 — Saisie Clavier Ultra-Rapide (`IpLineEdit` Smart Auto-Dot) . . . 2
1.4 Pilier 4 — Répétition Espacée & Gamification Style Minecraft . . . 3

2 Onglet 1 — Convertisseur Binaire 8-Bit & Inspecteur de Masque **4**
2.1 Pourquoi chaque élément est présent sur cet écran . . . 4
2.2 Le Raisonnement du Cours : L'Inspecteur de Masque & Complément . . . 4
2.2.1 1. Qu'est-ce qu'un Octet de Masque Valide ? (Règle de Contiguïté) . . . 4
2.2.2 2. Le Complément à 1 ($\sim X = 255 - X$) . . . 5
2.2.3 3. Le Pas du Sous-Réseau (*Magic Number* $= 256 - X$) . . . 6

3 Onglet 2 — Pratique Sous-Réseau, Classes & Tiroir Binaire 32-Bit **7**
3.1 Pourquoi la Barre de Classification (Classe & Type) est Placée en Haut . . . 7
3.2 Résolution Détaillée de l'Exemple du Cours (`154.4.32.0/19`) . . . 8

4 Onglet 3 — Atelier VLSM Express (Découpage à Masque Variable) **9**
4.1 Pourquoi le Découpage Classique (FLSM) ne Suffit Pas . . . 10
4.2 Les 3 Étapes Mathématiques Testées par l'Onglet 3 . . . 10
4.3 Corrigé Complet des 2 Exercices Officiels du Cours INPT . . . 10
4.3.1 Exercice du Cours 1 : Réseau `193.225.34.0/24` — Besoins `[44, 20, 100]` (Figure 5) . . . 10
4.3.2 Exercice du Cours 2 : Réseau `192.168.0.0/24` — Besoins `[100, 28, 4]` (Figure 6) . . . 10

5 Onglet 4 — Flash Drills : Fragmentation MTU, Routage LPM & Checksum **11**
5.1 Mini-Exercice 1 : Fragmentation IPv4 & MTU . . . 11
5.1.1 Le Raisonnement Mathématique du Cours . . . 11
5.2 Mini-Exercice 2 : Table de Routage, Longest Prefix Match (LPM) & AD . . . 13
5.2.1 L'Algorithme Exact de Sélection de Route d'un Routeur IP . . . 13
5.3 Mini-Exercice 3 : Somme de Contrôle de l'En-tête IPv4 (Checksum 16-Bit) . . . 15
5.3.1 Pourquoi et Comment Calculer le Checksum IPv4 (Figure 9) . . . 15

1 Philosophie Pédagogique : Pourquoi l'Application est Conçue Ainsi

Lorsqu'on prépare les examens et travaux dirigés de **Réseaux Informatiques (IPv4)** en cycle ingénieur, deux obstacles majeurs ralentissent l'apprentissage :

1. **La fatigue mécanique (Surcharge Cognitive)** : Dans un exercice de découpage VLSM classique, calculer 3 ou 4 sous-réseaux oblige à écrire 18 à 24 adresses IP complètes (Réseau, Masque, 1er Hôte, Dernier Hôte, Broadcast). Au bout de deux exercices, l'étudiant passe 80 % de son temps à recopier des octets répétitifs au lieu d'entraîner sa prise de décision mathématique.
2. **L'opacité du calcul binaire sur 32 bits** : Les calculatrices IP en ligne donnent directement la réponse finale sans montrer *comment* l'opérateur logique `ET` (`AND`) ou le complément à un (∼`Masque`) agissent bit par bit.

Pour résoudre ces deux problèmes, **IPv4 Subnet Toolkit v1.2.0** repose sur quatre piliers ergonomiques visibles partout dans l'application :

1.1 Pilier 1 — « Tester la Décision, Automatiser la Répétition »

Dans l'atelier VLSM (Onglet 3), l'application vous demande uniquement le **cœur mathématique** :

- Quel besoin traiter en premier ? (*Tri décroissant*).
- Quel préfixe `/CIDR` choisir ? (*Loi des puissances de 2 :* $2^h - 2 \geq N$).
- Quelle est la nouvelle **Adresse Réseau** ? (*Addition du pas* 2^h).

Dès que ces 3 décisions sont exactes (✓), l'application **auto-complète instantanément** la plage d'hôtes utilisables (`First Host` → `Last Host`), l'adresse de `Broadcast` et le masque décimal !

1.2 Pilier 2 — La Divulgation Progressive (Tiroir Binaire 32-Bit)

Dans l'onglet 2 (*Pratique Sous-Réseau*), le détail binaire sur 32 bits est placé dans un tiroir repliable (`Afficher le détail binaire (32 bits)`), fermé par défaut. Cela vous entraîne d'abord à calculer vite de tête (grâce au *Magic Number*), tout en vous permettant d'ouvrir le tiroir à tout moment pour vérifier les opérations bit à bit du cours (`IP ET Masque`, `IP ET` ∼`Masque`, bits empruntés b_S et bits d'hôtes b_H).

1.3 Pilier 3 — Saisie Clavier Ultra-Rapide (`IpLineEdit` Smart Auto-Dot)

Tous les champs d'adresse IP de l'application utilisent un composant intelligent :

- Appuyer sur **Espace** ou **Virgule** insère immédiatement un point `.` (ou saute par-dessus un point existant).
- Taper **3 chiffres** (ex: `192`) insère automatiquement le point `.`.
- **Nouveauté v1.2.0 (Auto-Dot ≥ 26)** : Puisqu'un octet IPv4 ne peut jamais dépasser `255`, **aucun octet valide à 3 chiffres ne peut commencer par un nombre entre 26 et 99** (car 260 > 255). Dès que vous tapez 2 chiffres supérieurs ou égaux à 26 (ex: `32`, `68`, `99`), l'application insère le point `.` toute seule sans attendre !

1.4 Pilier 4 — Répétition Espacée & Gamification Style Minecraft

- **Bannière d'astuces en bas de fenêtre** : Toutes les 12 secondes (ou au clic sur `Astuce`), une règle fondamentale du cours INPT s'affiche (`[COURS INPT]`, `[ASTUCE PRO]`, `[APP TIP]`).
- **Compteur de Série & Succès** : En haut à droite, le badge `Série : X` s'incrémente chaque fois que vous résolvez un exercice complet **sans avoir cliqué sur le bouton d'aide** `Show`, débloquant des bannières de progrès (*Maître du Masque*, *Architecte VLSM*, *Chirurgien MTU*, *Routeur Humain*, *Seigneur du Checksum*).

2 Onglet 1 — Convertisseur Binaire 8-Bit & Inspecteur de Masque

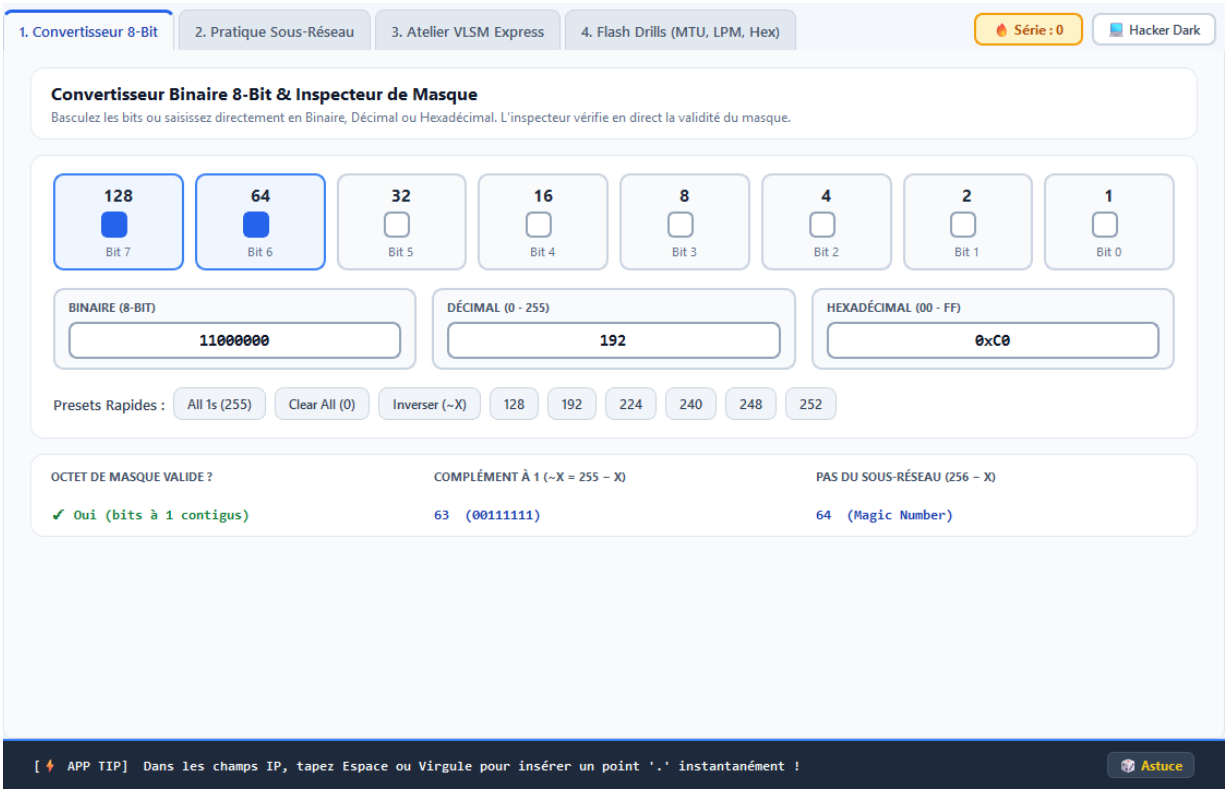


Figure 1: Onglet 1 en Mode Clair : l'octet `192` (`11000000`) est reconnu comme un octet de masque valide avec un pas (Magic Number) de `64`.

2.1 Pourquoi chaque élément est présent sur cet écran

1. **Les 8 Cartes de Bits** (`128, 64, 32, 16, 8, 4, 2, 1`) : Un octet IPv4 est un entier non signé sur 8 bits ($b_7b_6b_5b_4b_3b_2b_1b_0$) dont la valeur décimale est :

$$V = \sum_{i=0}^{7} b_i \cdot 2^i = 128b_7 + 64b_6 + 32b_5 + 16b_4 + 8b_3 + 4b_2 + 2b_1 + 1b_0$$

   Cliquer sur les cases permet de visualiser immédiatement comment les puissances de 2 s'additionnent.

2. **Synchronisation Temps Réel à 3 Voies (Binaire / Décimal / Hexadécimal)** : Utile à la fois pour l'adressage IPv4 (décimal/binaire) et pour le calcul de Checksum d'en-tête ou d'analyse Wireshark (hexadécimal `0x00` à `0xFF`).

3. **Les Presets Rapides** (`128, 192, 224, 240, 248, 252`) : Ce sont les valeurs d'octets de masque que vous rencontrerez dans 99 % des exercices de sous-réseaux et de VLSM.

2.2 Le Raisonnement du Cours : L'Inspecteur de Masque & Complément

En bas de l'onglet 1, la barre **Inspecteur de Masque & Complément** enseigne trois notions fondamentales du cours :

2.2.1 1. Qu'est-ce qu'un Octet de Masque Valide ? (Règle de Contiguïté)

Dans le cours de réseau, un masque de sous-réseau sert à séparer la **partie Réseau** (à gauche, bits à `1`) de la **partie Hôte** (à droite, bits à `0`). **Règle absolue** : Les bits à `1` d'un masque doivent être

strictement contigus en partant du bit de poids fort (Bit 7 = 128). Il est interdit d'avoir un `0` suivi d'un `1`.

Il n'existe donc que **9 valeurs décimales possibles** pour un octet de masque :

Bits à 1	Binaire (8 bits)	Valeur Décimale (X)	Pas du Sous-Réseau ($256 - X$)
0 bit	`00000000`	**0**	256
1 bit	`10000000`	**128**	128
2 bits	`11000000`	**192**	64
3 bits	`11100000`	**224**	32
4 bits	`11110000`	**240**	16
5 bits	`11111000`	**248**	8
6 bits	`11111100`	**252**	4
7 bits	`11111110`	**254**	2
8 bits	`11111111`	**255**	1



Figure 2: Onglet 1 en Mode Hacker Dark : l'octet `170` (`10101010`) alterne des `1` et des `0` → l'inspecteur indique immédiatement `Non (1 et 0 alternés)`.

2.2.2 2. Le Complément à 1 ($\sim X = 255 - X$)

Le complément à un inverse tous les bits (`0` ↔ `1`). Arithmétiquement, puisque la somme de tous les bits vaut 255 (`11111111`), inverser les bits revient à calculer :

$$\sim X = 255 - X$$

Sur la capture `192` (`11000000`), le complément à un vaut $255 - 192 = 63$ (`00111111`). Dans le cours, ce complément est utilisé pour :

- Extraire le **Host ID Interne** d'une machine via IP ET ($\sim$ Masque).

- Calculer l'adresse de **Broadcast** via Network OU ($\sim$ Masque) (c'est-à-dire additionner $\sim X$ à l'octet réseau ! Ex: $0 + 63 = 63$, $64 + 63 = 127$).

2.2.3 3. Le Pas du Sous-Réseau (*Magic Number* $= 256 - X$)

Le pas $256 - X$ donne l'espacement exact entre deux sous-réseaux consécutifs dans l'octet où le masque s'arrête. Avec $X = 192$, le pas vaut $256 - 192 = 64$: les sous-réseaux commencent donc aux multiples de 64 (`.0`, `.64`, `.128`, `.192`).

3 Onglet 2 — Pratique Sous-Réseau, Classes & Tiroir Binaire 32-Bit

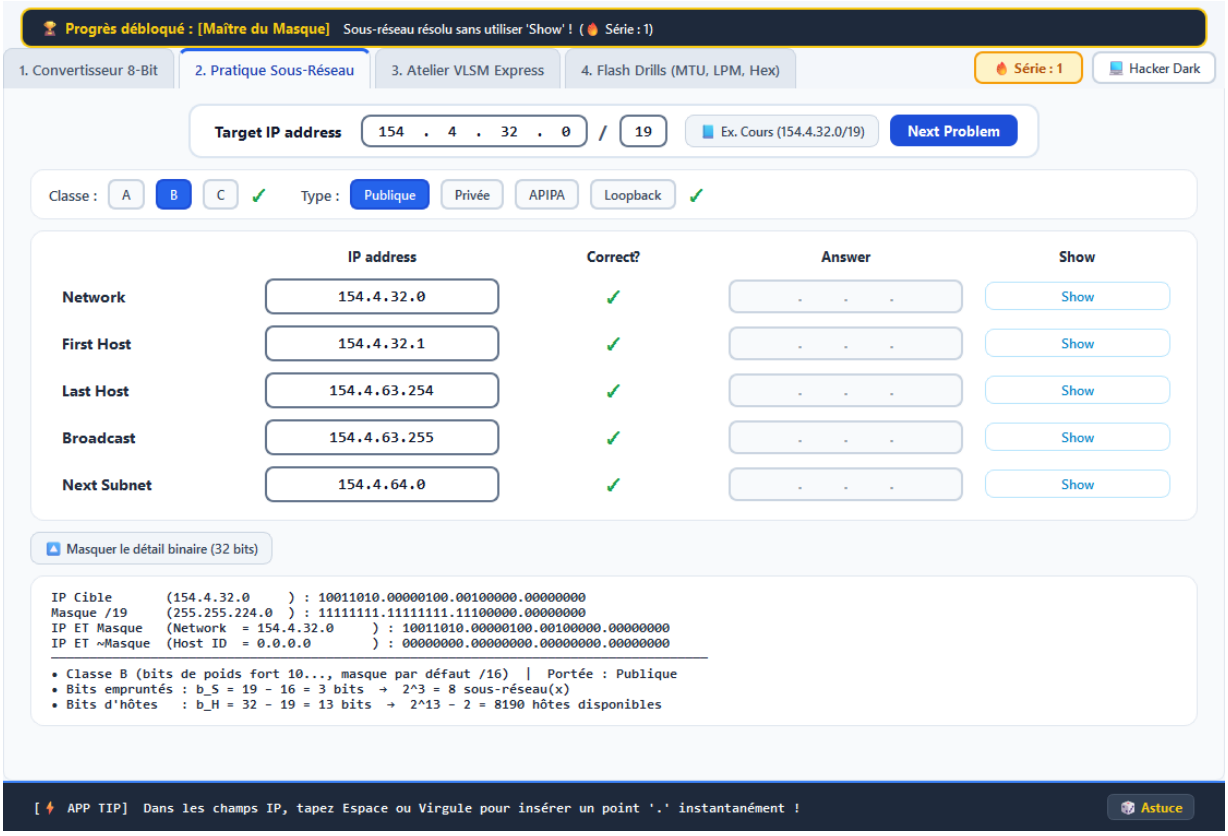


Figure 3: Onglet 2 avec l'exercice officiel du cours (`154.4.32.0/19`) entièrement résolu, le tiroir binaire 32 bits déployé et la bannière de succès débloquée.

3.1 Pourquoi la Barre de Classification (Classe & Type) est Placée en Haut

Avant même de calculer les plages d'un sous-réseau, le cours de réseau demande toujours de qualifier l'adresse IP cible selon deux axes :

Rappel du Cours INPT — Classes d'Adresses IPv4 (1er Octet)

- **Classe A** : `0` à `127` (Bit de poids fort `0...`, masque par défaut `/8` soit `255.0.0.0`).
- **Classe B** : `128` à `191` (Bits de poids fort `10...`, masque par défaut `/16` soit `255.255.0.0`).
- **Classe C** : `192` à `223` (Bits de poids fort `110...`, masque par défaut `/24` soit `255.255.255.0`).
- **Classe D** : `224` à `239` (Bits `1110...`, réservée au **Multicast** / diffusion de groupe).
- **Classe E** : `240` à `255` (Bits `11110...`, réservée pour un usage futur / expérimental).

Rappel du Cours INPT — Portée & Types d'Adresses Spéciales

- **Privée (RFC 1918)** — Non routable sur Internet (nécessite du NAT) :
  - Classe A : `10.0.0.0/8` (`10.0.0.0` à `10.255.255.255`)
  - Classe B : `172.16.0.0/12` (`172.16.0.0` à `172.31.255.255` — 16 réseaux de classe B)
  - Classe C : `192.168.0.0/16` (`192.168.0.0` à `192.168.255.255` — 256 réseaux de classe C)
- **Loopback (Boucle locale)** : `127.0.0.0/8` (ex: `127.0.0.1` pour tester la pile TCP/IP locale).
- **APIPA** : `169.254.0.0/16` (attribuée automatiquement par l'OS quand aucun serveur DHCP ne répond).
- **Publique** : Toute autre adresse unicast routable sur Internet (comme `154.4.32.0` sur la capture ci-dessus).

3.2 Résolution Détaillée de l'Exemple du Cours (`154.4.32.0/19`)

Regardons comment le tableau à 5 lignes et le tiroir binaire 32 bits (Figure 3) résolvent l'exemple officiel du cours :

1. **Identification de la Classe** : Le 1er octet est `154`. Comme $128 \leq 154 \leq 191$ (en binaire `10011010`, commence par `10`), c'est une **Classe B** (masque par défaut `/16`) et elle n'appartient pas à `172.16.0.0/12` ni `169.254.0.0/16` $\implies$ **Publique**.
2. **Analyse du Masque `/19`** :

$$19 \text{ bits à } 1 = \underbrace{11111111}_{255}.\underbrace{11111111}_{255}.\underbrace{11100000}_{224}.\underbrace{00000000}_{0} \implies \texttt{255.255.224.0}$$

   L'octet « intéressant » est le **3ème octet** (valeur 224). Son pas (*Magic Number*) vaut $256-224 = 32$.
3. **Calcul des 5 lignes du tableau** :
   - **Network Address** : Dans le 3ème octet, le multiple de 32 inférieur ou égal à 32 est 32 lui-même $\implies$ `154.4.32.0`.
   - **Next Subnet** : On ajoute le pas de 32 au 3ème octet : $32 + 32 = 64$ $\implies$ `154.4.64.0`.
   - **Broadcast Address** : Dernière adresse juste avant `154.4.64.0` $\implies$ `154.4.63.255`.
   - **First Host** : Network $+ 1$ $\implies$ `154.4.32.1`.
   - **Last Host** : Broadcast $- 1$ $\implies$ `154.4.63.254`.
4. **Lecture du Tiroir Binaire 32-Bit (en bas de la Figure 3)** :
   - **Bits empruntés** (b_S) : Par rapport au masque par défaut `/16` de la Classe B, on a emprunté $b_S = 19 - 16 = 3$ bits à la partie hôte, ce qui permet de créer $2^3 =$ **8 sous-réseaux**.
   - **Bits d'hôtes** (b_H) : Il reste $b_H = 32 - 19 = 13$ bits à `0` pour les machines, soit $2^{13} - 2 = 8192 - 2 =$ **8190 hôtes disponibles** par sous-réseau.
   - **Host ID Interne** : Calculé par IP ET ($\sim$ Masque). Ici, comme l'IP cible `154.4.32.0` est déjà l'adresse réseau, tous ses bits hôtes sont à `0` (`0.0.0.0`). Si l'IP cible était `154.4.35.12`, son Host ID interne serait `0.0.3.12`.



Figure 4: Onglet 2 en thème **Hacker Dark** (`#000000` / `#00FF66`) inspiré des terminaux réseau.

4 Onglet 3 — Atelier VLSM Express (Découpage à Masque Variable)

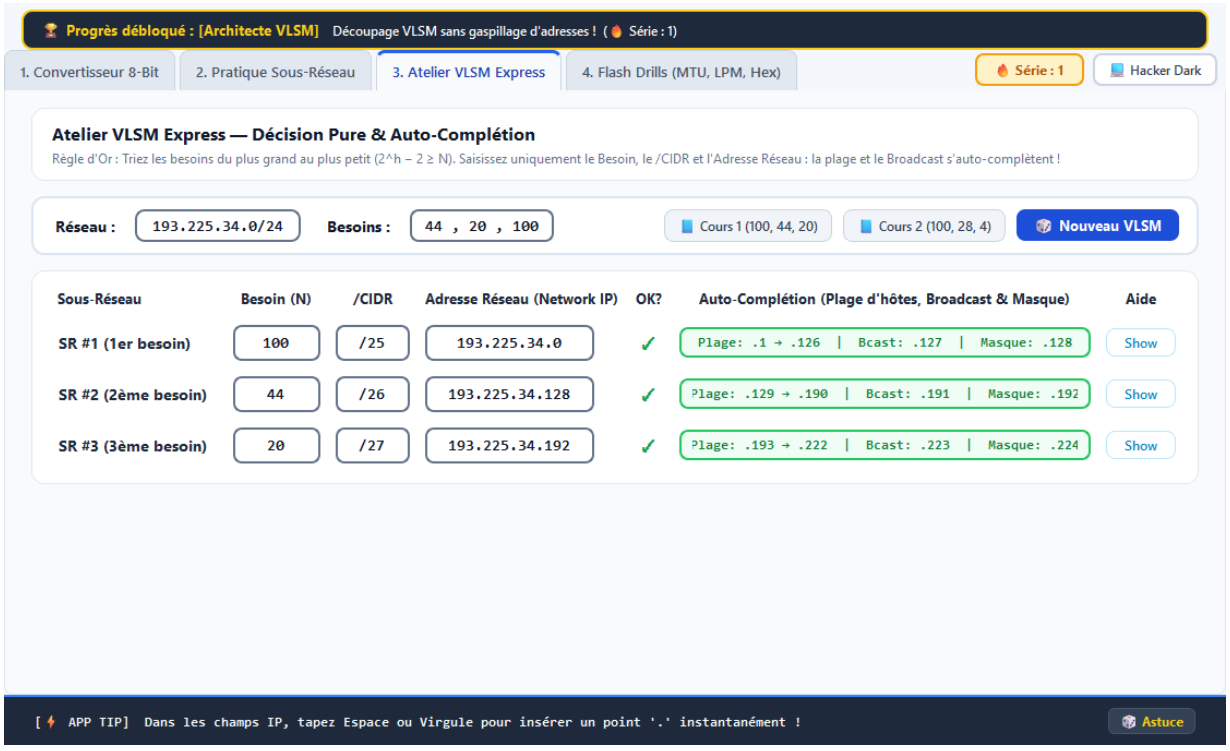


Figure 5: Onglet 3 — Résolution de l'Exercice Officiel du Cours 1 (`193.225.34.0/24` pour `44, 20, 100` machines) avec auto-complétion verte des plages.

4.1 Pourquoi le Découpage Classique (FLSM) ne Suffit Pas

En découpage fixe (*Fixed Length Subnet Mask*), tous les sous-réseaux ont la même taille. Si un service a besoin de 100 machines (`/25`, 126 places) et qu'une liaison point-à-point ou un petit bureau n'a besoin que de 4 machines, lui attribuer aussi un `/25` gaspille $126 - 4 = 122$ adresses IP ! Le **VLSM** (***Variable Length Subnet Mask***) résout ce problème en adaptant la longueur du masque `/CIDR` à chaque besoin réel.

4.2 Les 3 Étapes Mathématiques Testées par l'Onglet 3

1. **Colonne 1 — `Besoin (N)` (Le Tri Décroissant Obligatoire)** : Dans la barre du haut (Figure 5), les besoins sont donnés en désordre : `44 , 20 , 100`. Vous devez impérativement les saisir du **plus grand au plus petit** (`100`, puis `44`, puis `20`). *Pourquoi ?* Parce qu'en binaire, un sous-réseau de taille 2^h doit toujours démarrer sur une adresse multiple de 2^h. En plaçant les plus grands blocs en premier (Pas 128 → Pas 64 → Pas 32), chaque sous-réseau est naturellement aligné sur sa frontière binaire sans jamais chevaucher le précédent !

2. **Colonne 2 — `/CIDR` (Loi de dimensionnement $2^h - 2 \geq N$)** : Pour N machines, on cherche la plus petite puissance h telle que :

$$2^h - 2 \geq N \quad \text{(soit } 2^h \geq N + 2 \text{ en comptant Réseau et Broadcast)}$$

   Puis on déduit :

$$\text{CIDR} = 32 - h \quad \text{et} \quad \text{Pas (Taille du bloc)} = 2^h$$

3. **Colonne 3 — `Adresse Réseau (Network IP)`** : Le 1er sous-réseau démarre à l'adresse de base. Le sous-réseau suivant démarre à :

$$\text{Network}_{k+1} = \text{Network}_k + 2^{h_k}$$

4.3 Corrigé Complet des 2 Exercices Officiels du Cours INPT

4.3.1 Exercice du Cours 1 : Réseau `193.225.34.0/24` — Besoins `[44, 20, 100]` (Figure 5)

Rang	Besoin N	Calcul $2^h - 2 \geq N$	/CIDR	Masque	Adresse Réseau	Plage Hôtes & Broadcast
SR #1	**100**	$2^7 - 2 = 126 \geq 100$ ($h = 7$, Pas=128)	**/25**	`.128`	`193.225.34.0`	`.1` → `.126` | Bcast `.127`
SR #2	**44**	$2^6 - 2 = 62 \geq 44$ ($h = 6$, Pas=64)	**/26**	`.192`	`193.225.34.128`	`.129` → `.190` | Bcast `.191`
SR #3	**20**	$2^5 - 2 = 30 \geq 20$ ($h = 5$, Pas=32)	**/27**	`.224`	`193.225.34.192`	`.193` → `.222` | Bcast `.223`

Adresses restantes non allouées : de `193.225.34.224` à `.255` (soit 32 adresses libres).

4.3.2 Exercice du Cours 2 : Réseau `192.168.0.0/24` — Besoins `[100, 28, 4]` (Figure 6)

Rang	Besoin N	Calcul $2^h - 2 \geq N$	/CIDR	Masque	Adresse Réseau	Plage Hôtes & Broadcast
SR #1	**100**	$2^7 - 2 = 126 \geq 100$ ($h = 7$, Pas=128)	**/25**	`.128`	`192.168.0.0`	`.1` → `.126` | Bcast `.127`
SR #2	**28**	$2^5 - 2 = 30 \geq 28$ ($h = 5$, Pas=32)	**/27**	`.224`	`192.168.0.128`	`.129` → `.158` | Bcast `.159`
SR #3	**4**	$2^3 - 2 = 6 \geq 4$ ($h = 3$, Pas=8)	**/29**	`.248`	`192.168.0.160`	`.161` → `.166` | Bcast `.167`



Figure 6: Onglet 3 en Mode Hacker Dark — Résolution de l'Exercice Officiel du Cours 2 (`192.168.0.0/24` pour `100, 28, 4` machines).

5 Onglet 4 — Flash Drills : Fragmentation MTU, Routage LPM & Checksum

Pour éviter de surcharger l'écran, l'onglet 4 dispose d'une barre de sélection en haut qui permet de basculer entre les **3 grands exercices classiques d'examen** du cours de réseau.

5.1 Mini-Exercice 1 : Fragmentation IPv4 & MTU

5.1.1 Le Raisonnement Mathématique du Cours

Lorsqu'un routeur doit transmettre un paquet IPv4 de taille totale `Total Length` sur une liaison dont la taille maximale de trame est `MTU` (*Maximum Transmission Unit*, typiquement 1500 octets sur Ethernet), si Total Length > MTU, le paquet doit être découpé en plusieurs fragments IPv4 :

1. **Taille de l'En-Tête (`Header Length`)** : Le champ `IHL` indique le nombre de mots de 32 bits (4 octets) de l'en-tête.

$$L_{\text{en-tête}} = \text{IHL} \times 4 \quad (\text{Ex: IHL} = 5 \implies 20 \text{ octets})$$

   Important : Chaque fragment créé recevra sa propre copie de cet en-tête de 20 octets !

2. **Données Originales à Transporter** :

$$D_{\text{initial}} = \text{Total Length} - L_{\text{en-tête}} = 4000 - 20 = 3980 \text{ octets}$$

3. **Données Utiles Max par Fragment (Règle du Multiple de 8)** : Dans l'en-tête IPv4, le champ `Fragment Offset` est codé sur 13 bits alors que la longueur totale est sur 16 bits

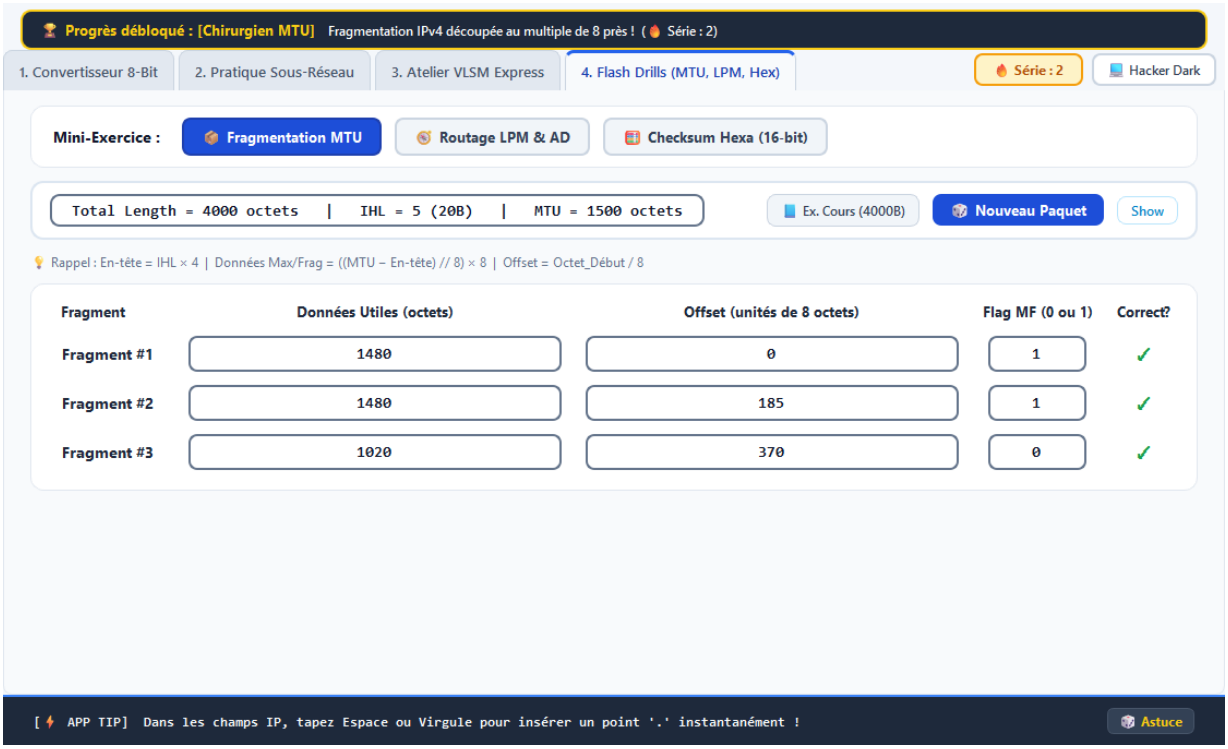


Figure 7: Onglet 4 (Vue 1) — Exercice officiel de Fragmentation MTU (`Total Length = 4000 octets`, `IHL = 5`, `MTU = 1500 octets`).

($2^{16}/2^{13} = 8$). L'Offset compte donc par **blocs de 8 octets**. Par conséquent, la quantité de données de chaque fragment (sauf le dernier) **doit être divisible par 8** :

$$D_{\max} = \left\lfloor \frac{\text{MTU} - L_{\text{en-tête}}}{8} \right\rfloor \times 8 = \left\lfloor \frac{1500 - 20}{8} \right\rfloor \times 8 = 185 \times 8 = 1480 \text{ octets}$$

4. **Calcul des 3 Colonnes de la Figure 7** :
   - **Fragment #1** : Transporte **1480** octets de données (octets 0 à 1479). Début = octet 0 $\Longrightarrow$ Offset = 0/8 = **0**. Il reste 3980 − 1480 = 2500 octets $\Longrightarrow$ bit **MF** (***More Fragments***) = **1**.
   - **Fragment #2** : Transporte **1480** octets de données (octets 1480 à 2959). Début = octet 1480 $\Longrightarrow$ Offset = 1480/8 = **185**. Il reste 2500 − 1480 = 1020 octets $\Longrightarrow$ **MF** = **1**.
   - **Fragment #3** : Transporte les **1020** derniers octets (octets 2960 à 3979). Début = octet 2960 $\Longrightarrow$ Offset = 2960/8 = **370**. C'est le dernier fragment $\Longrightarrow$ **MF** = **0**.

5.2 Mini-Exercice 2 : Table de Routage, Longest Prefix Match (LPM) & AD

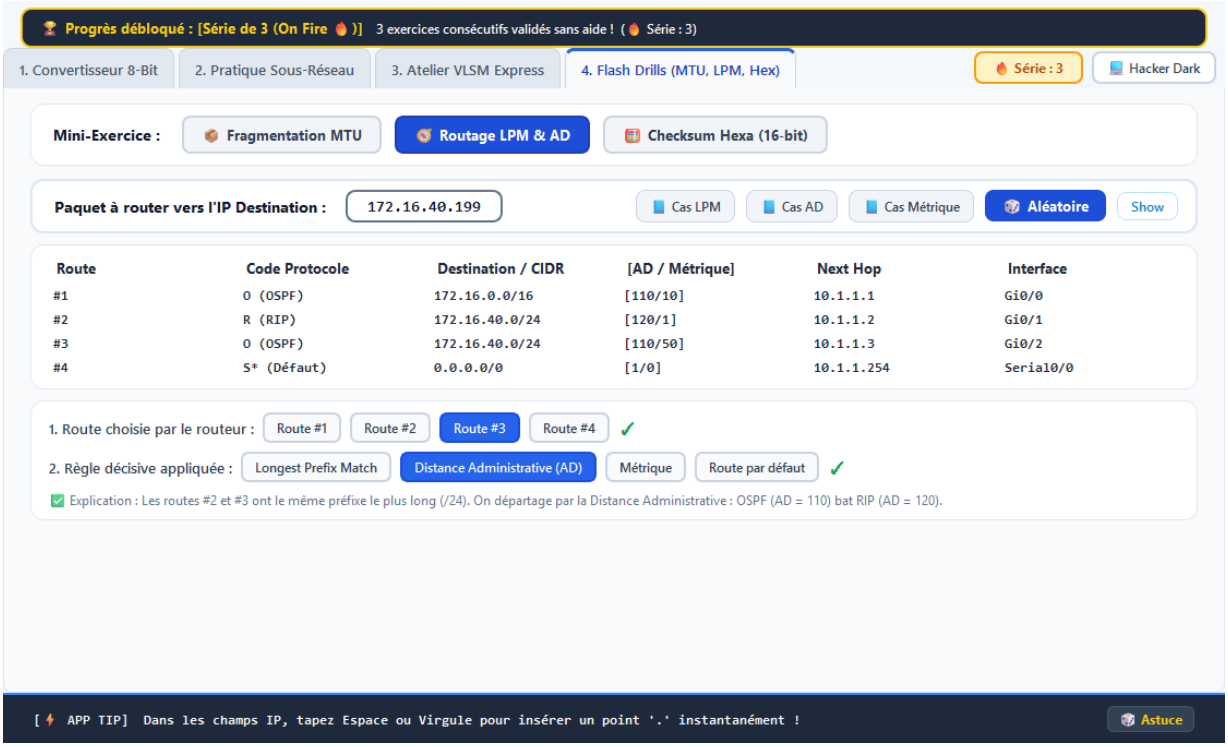


Figure 8: Onglet 4 (Vue 2) — Table de routage et sélection de la route gagnante et de la règle décisive.

5.2.1 L'Algorithme Exact de Sélection de Route d'un Routeur IP

Pour acheminer un paquet vers une adresse `IP Destination` (ex: `172.16.30.74` sur la Figure 8), le routeur n'examine jamais les routes au hasard. Il applique un filtre suivi de **3 règles hiérarchiques strictes** :

1. **Étape 0 — Filtrage d'Appartenance** : Le routeur élimine toutes les lignes de la table dont le préfixe réseau ne contient pas l'IP de destination. *Piège classique d'examen présent dans l'application* : Si une route est en `192.168.10.0/26` (qui ne couvre que `.0` à `.63`) et que le paquet va vers `192.168.10.80`, cette route `/26` ne correspond **pas** !

2. **Règle 1 — Longest Prefix Match (LPM / Plus Long Préfixe)** : Parmi toutes les routes qui correspondent à l'IP de destination, le routeur choisit **toujours celle qui possède le masque `/CIDR` le plus grand** (la route la plus spécifique : `/26 > /24 > /16 > /8 > /0`), quelle que soit sa distance administrative !

3. **Règle 2 — Distance Administrative (AD)** : Si (et seulement si) plusieurs routes correspondantes ont **exactement le même préfixe `/CIDR` le plus long**, le routeur départage selon la fiabilité du protocole (plus petite AD gagne) :

Protocole / Origine de la Route	Code Table	Distance Administrative (AD)
Réseau Directement Connecté	`C`	**0**
Route Statique	`S`	**1**
OSPF (état de liens)	`O`	**110**
RIP (vecteur de distance)	`R`	**120**

Sur la Figure 8, les routes #2 (`R 172.16.30.0/24 [120/1]`) et #3 (`O 172.16.30.0/24 [110/50]`) ont le même masque `/24`. Même si RIP affiche une métrique de `1` contre `50` pour OSPF, le routeur compare d'abord l'**AD** : OSPF (110) < RIP (120) $\Longrightarrow$ **Route #3 gagne par Distance Administrative** !

4. **Règle 3 — Métrique** : Si plusieurs routes ont le **même** `/CIDR` et la **même AD** (même protocole), le routeur choisit la plus petite métrique.

5. **Cas de la Route par Défaut (`0.0.0.0/0`, `S*`)** : Son masque est `/0` (0 bit vérifié : elle correspond à 100 % des adresses IPv4, mais avec le préfixe le plus court possible). Elle n'est donc utilisée que si aucune autre route de la table ne correspond à la destination.

5.3 Mini-Exercice 3 : Somme de Contrôle de l'En-tête IPv4 (Checksum 16-Bit)



Figure 9: Onglet 4 (Vue 3) en Mode Hacker Dark — Calcul en 3 étapes du Checksum 16 bits sur l'exemple officiel du cours.

5.3.1 Pourquoi et Comment Calculer le Checksum IPv4 (Figure 9)

L'en-tête IPv4 (20 octets pour IHL = 5) contient un champ `Header Checksum` de 16 bits qui protège l'en-tête contre les erreurs de transmission et qui doit être recalculé à chaque routeur (car le champ `TTL` diminue de 1 à chaque saut).

Le cours décompose ce calcul en **3 étapes**, exactement reproduites par les 3 champs de saisie de la Figure 9 :

1. **Étape 1 — Somme Hexadécimale Brute des 10 Mots de 16 Bits** : On place le champ Checksum (6ème mot) à `0000` et on additionne les mots de 16 bits :

$$\texttt{4500} + \texttt{0054} + \texttt{0000} + \texttt{4000} + \texttt{4001} + \texttt{0000} + \texttt{C0A8} + \texttt{0068} + \texttt{C0A8} + \texttt{0001} = \texttt{0x2470E}$$

2. **Étape 2 — Rabattement de la Retenue (*End-Around Carry* sur 16 bits)** : Le résultat `0x2470E` dépasse 16 bits (4 chiffres hexa). Le chiffre de dépassement à gauche est `2` (poids 2×2^{16}). On l'additionne aux 16 bits de droite (`0x470E`) :

$$\texttt{0x470E} + \texttt{0x0002} = \texttt{0x4710}$$

3. **Étape 3 — Complément à Un Final (~ Somme & `0xFFFF`)** : On inverse tous les bits (ou de manière équivalente en hexadécimal, on soustrait chaque chiffre à `F`, soit `0xFFFF` − `0x4710`) :

$$\begin{array}{rcccc} & \texttt{F} & \texttt{F} & \texttt{F} & \texttt{F} \\ - & \texttt{4} & \texttt{7} & \texttt{1} & \texttt{0} \\ \hline = & \texttt{B} & \texttt{8} & \texttt{E} & \texttt{F} \end{array} \quad \Longrightarrow \text{ Checksum final} = \texttt{0xB8EF}$$

Vérification par le récepteur : À la réception du paquet, le routeur additionne les 10 mots **en incluant** le Checksum `0xB8EF`. Après rabattement de la retenue, il obtient `0x4710` + `0xB8EF` = `0xFFFF` (que des `1`, dont le complément à un vaut `0x0000`), preuve que l'en-tête est intact !